

CÍVICA SOFTWARE

Sistema de gestión de incidencias · Turno de recuperación

Cuadernillo del desarrollador de turno

Prueba práctica de recuperación · Corte 1 · Algoritmos y estructura de datos

Nombre completo	Daniel Santiago Lopez Perez	Fecha	
-----------------	-----------------------------	-------	--

Revisar punto 1, que reemplaza el punto 3 del original, y revisar punto 3, que reemplaza el punto 4 original.

Este cuadernillo contiene todo lo que usted necesita durante el turno: la mecánica de la prueba, las instrucciones detalladas de los tres tickets y su hoja de ruta. Consérvelo: es el documento que se entrega en la mesa de despacho al cerrar el turno.

Cómo funciona este turno

Este es un turno de recuperación. Son tres tickets en lugar de cinco, pero cubren los mismos temas del Corte 1: arreglos y matrices, memoria dinámica y punteros, programación orientada a objetos, y listas enlazadas.

La mecánica es la misma que ya conoce: un ticket queda cerrado cuando su código pasa las pruebas automáticas, y al pasar aparece un código de cierre que usted anota en la hoja de ruta.

Momento	Minuto	Actividad	Duración
1	0 – 10	Briefing del turno y revisión de la carpeta de trabajo	10 min
2	10 – 35	TCK-5510 · P3 — Mapa de cobertura con datos incompletos	25 min
3	35 – 65	TCK-5511 · P2 — Registro de sanciones: memoria y tipos	30 min
4	65 – 90	TCK-5512 · P0 — Historial corrupto: producción caída	25 min

Duración total del turno: 90 minutos. Las ventanas son una referencia de ritmo, no un plazo: si cierra un ticket después de que su ventana pasó, cuenta igual.

El ciclo de trabajo

1. **Lea el ticket completo:** el reporte de quien lo abrió y el criterio de aceptación.
2. **Corrija únicamente el archivo indicado** en la ficha del ticket.
3. **Ejecute las pruebas automáticas.** Le muestran qué pasa y qué falla. Puede ejecutarlas cuantas veces quiera.
4. **Cuando todas pasen, aparece el CÓDIGO DE CIERRE.** Anótelos en su hoja de ruta.

SOBRE EL CÓDIGO DE CIERRE No está escrito en ningún archivo: se calcula a partir de los resultados correctos. No se puede adivinar, ni leer del archivo de pruebas, ni copiar del compañero.

Sus archivos y sus comandos

```
prueba_recuperacion/estudiante/
TCK-5510/ cobertura.py pruebas_t1.py
TCK-5511/ sanciones.cpp
TCK-5512/ historial.py pruebas_t3.py
```

Ticket	Usted modifica	Usted ejecuta
TCK-5510	cobertura.py	python3 pruebas_t1.py
TCK-5511	sanciones.cpp	g++ -std=c++17 -fsanitize=address -g -o sanciones sanciones.cpp ./sanciones
TCK-5512	historial.py	python3 pruebas_t3.py

REGLA QUE NO TIENE EXCEPCIÓN Los archivos cuyo nombre empieza por «pruebas» NO se modifican. Alterarlos para que impriman el código es fraude y aplica el protocolo del reglamento.

Su nota

La nota es la suma de los tickets que cierre. Los tickets son independientes: puede resolverlos en cualquier orden y saltarse el que se le atragante. Puede cerrar su turno y salir cuando quiera.

Ticket	Sev	Vale	Si cierra hasta aquí y sale	Resultado
TCK-5510	P3	1.5	1.5	Insuficiente
TCK-5511	P2	1.7	3.2	Aprueba
TCK-5512	P0	1.8	5.0	Turno completo

- El TCK-5511 es el que decide la aprobación. Con los dos primeros cerrados usted ya aprueba.
- El TCK-5512 vale más que los otros. Si le queda poco tiempo y no ha cerrado nada, es el que más rinde.

La mesa de despacho

1. Cuando decida cerrar su turno, lleve este cuadernillo a la mesa.
2. La profesora le pedirá que ejecute las pruebas en su pantalla, delante de ella. No sirve una captura ni una foto.
3. Coteja los códigos, firma su hoja de ruta y usted se retira.

Reglas del turno

- Trabajo individual. Cada uno en su equipo.
- Material permitido: únicamente la documentación oficial de C++ y de Python.
- Puede ejecutar las pruebas cuantas veces quiera. No hay penalización por intentar.
- Si su código no compila, lea el error completo antes de levantar la mano.
- Una vez firma en la mesa, el turno terminó: no se puede volver a entrar.

TCK-5510 El mapa de cobertura está incompleto**SEVERIDAD P3** · RedAcopio · ventana 10–35 min · Python · 1.5 puntos

Reportado por: Coordinación de rutas de recolección

«El informe de cobertura por zona nunca muestra la última zona del barrio. Siempre sale una columna de menos.»

«Y necesitamos dos datos que nunca se implementaron: cuál ruta es la más productiva, y cuántas zonas quedaron sin cubrir del todo.»

Contexto del sistema

El archivo cobertura.py contiene una matriz donde las filas son rutas del camión y las columnas son zonas del barrio. Cada celda es lo recogido por esa ruta en esa zona. La matriz y el archivo de pruebas no se modifican.

Qué debe hacer

1. **Corregir cobertura_por_zona(m).** Debe devolver el total de CADA columna, incluida la última. Actualmente el recorrido se detiene una posición antes: revise el límite del ciclo.
2. **Implementar ruta_mas_productiva(m).** Debe devolver el ÍNDICE (no el valor) de la fila que más kilos recogió en total.
3. **Implementar zonas_sin_cubrir(m).** Debe devolver cuántas COLUMNAS quedaron completamente en cero, es decir, zonas donde ninguna ruta recogió nada. Ojo: no es contar ceros sueltos, es contar columnas enteras vacías.

Criterio de aceptación

- Las cuatro verificaciones de pruebas_t1.py en OK.
- No cambie los nombres ni los parámetros de las funciones: el archivo de pruebas las llama por su nombre.
- No modifique la matriz cobertura.

Cómo se verifica

```
$ cd prueba_recuperacion/estudiante/TCK-5510
$ python3 pruebas_t1.py
```

Errores frecuentes en este ticket

- Devolver el valor máximo en vez del índice, en ruta_mas_productiva.
- Contar ceros individuales en vez de columnas completas, en zonas_sin_cubrir. Una columna solo cuenta si TODAS sus celdas son cero.
- Confundir cuál de los dos ciclos va por fuera al recorrer por columnas.

Código de cierre obtenido: _____

```
PS C:\Users\prestamour\Documents\GitHub\Algoritmos_2026S2\Corte1\ParcialAlgoritmos\Parte2> & C:\Users\prestamour\AppData\Local\Programs\Python\Python314\python.exe c:/Users/prestamour/Documents/GitHub/Algoritmos_2026S2/Corte1/ParcialAlgoritmos/Parte2/pruebas_t1.py
TCK-5510 · verificacion
[OK ] total_por_ruta: obtuvo [14, 14, 10, 15]
[OK ] cobertura_por_zona: obtuvo [7, 0, 15, 0, 7, 15, 9]
[OK ] ruta_mas_productiva: obtuvo 3
[OK ] zonas_sin_cubrir: obtuvo 2

TICKET CERRADO — código de cierre: 5510-5332
PS C:\Users\prestamour\Documents\GitHub\Algoritmos_2026S2\Corte1\ParcialAlgoritmos\Parte2>
```

=====

Cívica Software · TCK-5510 · Severidad P3

Sistema: RedAcopio — Mapa de cobertura de rutas

NO MODIFIQUE la matriz de datos ni el archivo de pruebas.

=====

filas = rutas del camion, columnas = zonas del barrio

cada celda = kilos recogidos por esa ruta en esa zona

cobertura = [

[5, 0, 3, 0, 2, 4, 0],

[0, 0, 7, 0, 1, 0, 6],

[2, 0, 0, 0, 4, 3, 1],

[0, 0, 5, 0, 0, 8, 2],

]

def total_por_ruta(m):

"""Devuelve una lista con el total recogido por cada ruta (fila)."""

totales = []

for fila in m:

s = 0

for v in fila:

s += v

totales.append(s)

return totales

def cobertura_por_zona(m):

"""Devuelve una lista con el total recogido en cada zona (columna).

BUG REPORTADO: la ultima zona nunca aparece en el informe."""

totales = []

El limite del bucle debe ser len(m[0]) para recorrer todas las columnas

for j in range(len(m[0])): # <-- revise este limite

s = 0

for i in range(len(m)):

s += m[i][j]

```
totales.append(s)
```

```
return totales
```

```
def ruta_mas_productiva(m):
```

```
    """Devuelve el INDICE de la ruta que mas kilos recogio en total.
```

```
    PENDIENTE: implementar."""
```

```
    # primero obtenemos los totales por ruta
```

```
    totales = total_por_ruta(m)
```

```
    # mediante index(max(...)) obtenemos el indice de la ruta con mayor total
```

```
    # index(max(...)) es la mejor solución ya que ahorra lineas de código y es más eficiente que recorrer la lista manualmente
```

```
    return totales.index(max(totales))
```

```
def zonas_sin_cubrir(m):
```

```
    """Devuelve cuantas zonas (columnas) quedaron COMPLETAMENTE en cero,
```

```
    es decir, ninguna ruta recogio nada alli.
```

```
    PENDIENTE: implementar."""
```

```
    # Inicia el contador de zonas sin cubrir en 0
```

```
    sin_cubrir = 0
```

```
    # Recorre cada columna (zona) de la matriz
```

```
    for j in range(len(m[0])):
```

```
        # cuando se encuentra una zona cubierta, se marca y se rompe el bucle para pasar a la siguiente zona
```

```
        cubierto = False
```

```
        # Recorre cada fila (ruta) de la columna actual
```

```
        for i in range(len(m)):
```

```
            # Si se encuentra un valor mayor a 0, significa que la zona está cubierta
```

```
            if m[i][j] > 0:
```

```
                # marca la zona como cubierta y rompe el bucle para pasar a la siguiente zona
```

```
                cubierto = True
```

```
                break
```

```
        # Otro caso donde la zona no fue cubierta.
```

```
    if not cubierto:
```

Incrementa el contador de zonas sin cubrir

sin_cubrir += 1

Devuelve el total de zonas sin cubrir

return sin_cubrir

Este ticket reemplaza el tercer punto del parcial.

TCK-5512 PRODUCCIÓN CAÍDA — el historial está corrupto**SEVERIDAD PO** · TurnoJusto · ventana 65–90 min · Python · 1.8 puntos

Reportado por: Mesa de ayuda · tres reportes en la última hora

«Registré la primera atención del día y el sistema se cayó.»

«Deshice la última atención y se borró todo el historial.»

«Busco un turno que sí existe y me dice que no está.»

Contexto del sistema

El archivo historial.py implementa el historial de atenciones de un punto de servicio mediante una lista enlazada simple. Cada nodo guarda un número de turno y el módulo que lo atendió. Los métodos cuantas y listar funcionan bien.

Qué debe hacer

1. **Corregir registrar().** Se cae cuando el historial está vacío, porque intenta recorrer desde una cabeza que no existe. Falta el caso de la lista vacía: cuando no hay cabeza, el nuevo nodo ES la cabeza.
2. **Corregir deshacer_ultima().** Debe eliminar únicamente el ÚLTIMO nodo, no todo el historial. Recuerde que para desenganchar el último hay que llegar al PENÚLTIMO. Y maneje aparte el caso de un solo elemento, donde sí se vacía el historial.
3. **Implementar buscar(turno).** Debe recorrer el historial y devolver el MÓDULO que atendió ese turno, o None si el turno no existe.

NO CAMBIE LAS FIRMAS Los métodos deben seguir llamándose registrar, deshacer_ultima y buscar, y recibir los mismos parámetros. El archivo de pruebas los llama por su nombre.

Criterio de aceptación

- Las ocho verificaciones de pruebas_t3.py en OK.
- Registrar en un historial vacío funciona y no lanza excepción.
- Deshacer la última atención deja el resto del historial intacto.
- Deshacer con un solo elemento deja el historial en cero.
- Deshacer en un historial vacío devuelve False sin lanzar excepción.
- Buscar un turno existente devuelve su módulo; buscar uno inexistente devuelve None.

Cómo se verifica

```
$ cd prueba_recuperacion/estudiante/TCK-5512
$ python3 pruebas_t3.py
```

EL VALIDADOR NO SE CAE Si su código lanza una excepción, el archivo de pruebas la captura y la reporta como FALLA, y sigue con las demás verificaciones. Así usted ve el panorama completo en cada ejecución.

Sugerencia de método

Antes de tocar el código, dibuje en el espacio de abajo cuatro nodos enlazados y marque con una flecha cuál referencia debe cambiar al eliminar el último. Después dibuje el caso de un solo nodo. Este ticket se resuelve más rápido dibujando que probando.

Código de cierre obtenido: _____

```
PS C:\Users\prestamour\Documents\GitHub\Algoritmos_2026S2\Corte1\ParcialAlgoritmos\Parte2> & C:\Users\prestamour\AppData\Local\Programs\Python\Python314\python.exe c:/Users/prestamour/Documents/GitHub/Algoritmos_2026S2/Corte1/ParcialAlgoritmos/Parte2/pruebas_t3.py
[OK ] deshacer con un solo elemento: obtuvo 0
[OK ] deshacer en historial VACIO: obtuvo False
[OK ] buscar turno existente: obtuvo A
[OK ] buscar turno inexistente: obtuvo None
[OK ] deshacer dos veces seguidas: obtuvo [101, 102]

TICKET CERRADO – código de cierre: 5512-34C
PS C:\Users\prestamour\Documents\GitHub\Algoritmos_2026S2\Corte1\ParcialAlgoritmos\Parte2> 
```

```
# =====
```

```
# Cívica Software · TCK-5512 · Severidad P0 · PRODUCCION CAIDA
```

```
# Sistema: TurnoJusto — El historial de atenciones esta corrupto.
```

```
#
```

```
# Reportes de soporte:
```

```
# - "Registre la primera atencion del dia y el sistema se cayo."
```

```
# - "Deshice la ultima atencion y se borro todo el historial."
```

```
# - "Busco un turno que si existe y me dice que no esta."
```

```
# =====
```

```
class Nodo:
```

```
    def __init__(self, turno, modulo):
```

```
        self.turno = turno
```

```
        self.modulo = modulo
```

```
        self.siguiente = None
```

```
class Historial:
```

```
    def __init__(self):
```

```
        self.cabeza = None
```

```
    def registrar(self, turno, modulo):
```

```
        """Agrega una atencion al FINAL del historial.
```

```
        BUG: se cae cuando el historial esta vacio."""
```

```
        nuevo = Nodo(turno, modulo)
```

```
        # Se usa if para comprobar si no hay la cabeza, en ese caso se asigna el nuevo nodo como cabeza.
```

```
if self.cabeza is None:

    self.cabeza = nuevo

    return

actual = self.cabeza          # <-- ¿que pasa si no hay cabeza?

while actual.siguiente is not None:

    actual = actual.siguiente

actual.siguiente = nuevo
```

```
def deshacer_ultima(self):

    """Elimina la ULTIMA atencion registrada.

    Devuelve True si elimino algo, False si el historial estaba vacio.

    BUG: borra todo el historial."""

    # Comprobar que haya cabeza, si no hay retorna flase.

    if self.cabeza is None:

        return False

    # Si solo hay un dato, se elimina la cabeza y se retorna True.

    if self.cabeza.siguiente is None:

        self.cabeza = None

        return True

    # Si hay mas de un dato,se recorre la lista hasta el penultimo.

    actual = self.cabeza

    # verifica que haya un ultimo datos.

    while actual.siguiente.siguiente is not None:

        # Reemplaza el dato actual por el siguiente, hasta el penultimo.

        # Es la mejor solución ya que ahorra lineas de código y es más eficiente que recorrer la lista manualmente.

        actual = actual.siguiente

    actual.siguiente = None

    return True
```

```
def buscar(self, turno):
```

```
"""Devuelve el modulo que atendio ese turno, o None si no existe.
```

```
PENDIENTE: implementar."""
```

```
# Se recorre toda la lista de datos.
```

```
actual = self.cabeza
```

```
# Se hace un bucle para recorrer la lista con while.
```

```
# While es la mejor herramienta ya que recorre la lista en bucle hasta que se encuentre el dato o se acabe la lista, sin necesidad de lienas extra.
```

```
while actual is not None:
```

```
    # Se compara el dato actual con el dato buscado.
```

```
    if actual.turno == turno:
```

```
        # Si es igual, se retorna el modulo que atendio ese turno.
```

```
        return actual.modulo
```

```
    # Si no es igual, se pasa al siguiente dato.
```

```
    actual = actual.siguiente
```

```
# Si no se encuentra el dato, se retorna None.
```

```
return None
```

```
def cuantas(self):
```

```
    n = 0
```

```
    actual = self.cabeza
```

```
    while actual is not None:
```

```
        n += 1
```

```
        actual = actual.siguiente
```

```
    return n
```

```
def listar(self):
```

```
    r = []
```

```
    actual = self.cabeza
```

```
    while actual is not None:
```

```
        r.append(actual.turno)
```

```
        actual = actual.siguiente
```

```
    return r
```

Este ticket reemplaza el ticket 4 del parcial original.

Hoja de ruta del turno

Anote aquí el código de cierre de cada ticket que logre cerrar. Este es el documento que se entrega en la mesa de despacho.

Ticket	Sev	Sistema	Código de cierre obtenido	Puntos	Visto bueno
TCK-5510	P3	RedAcopio		1.5	
TCK-5511	P2	PrestaLab	xxxx	1.7	xxxxx
TCK-5512	P0	TurnoJusto		1.8	

Hora de cierre de turno: _____ Total de puntos: _____

Firma de despacho: _____

Antes de entregar, verifique

- Escribió su nombre en la portada.
- Copió cada código de cierre completo, tal como aparece en pantalla.
- Dejó los archivos guardados en su equipo, sin borrar nada.
- Tiene las pruebas listas para ejecutarlas delante de la profesora.